ARIA

Adaptive Review Intelligence Architecture

System Design Document

Version 1.0

Project:	Automatic Code Reviewer
Theme:	Multi-LLM Intelligent Code Review
Date:	April 2026
Course:	Design Lab

An intelligent automatic code review system that leverages multiple LLM families to produce comprehensive, reliable, and actionable code review reports.

Contents

1 Executive Summary **3**
- 1.1 Key Innovation: The Model Debate Protocol . . . 3
- 1.2 Design Goals . . . 3

2 Problem Statement **3**
- 2.1 The Industry Crisis . . . 3
- 2.2 Problem Definition . . . 4

3 Literature Review & Industry Analysis **4**
- 3.1 Current Tool Landscape . . . 4
- 3.2 Key Industry Trends . . . 4
- 3.3 Gap Analysis . . . 5

4 System Architecture **5**
- 4.1 High-Level Pipeline . . . 5
- 4.2 Detailed Component Architecture . . . 6
  - 4.2.1 Stage 1: Repository Ingestion Engine . . . 6
  - 4.2.2 Stage 2: Specialized Agent Swarm . . . 7
  - 4.2.3 Stage 3: Model Debate Protocol . . . 7
  - 4.2.4 Stage 4: LangGraph State Machine . . . 8
  - 4.2.5 Stage 5: Report Generator . . . 8

5 Technology Stack **9**

6 Data Flow Diagram **10**

7 The Model Debate Protocol — Detailed Design **10**
- 7.1 Motivation . . . 10
- 7.2 Protocol Steps . . . 11
- 7.3 Why This Works . . . 11

8 AI-Slop Detection — Novel Contribution **11**
- 8.1 Detection Heuristics . . . 11
- 8.2 AI-Slop Score . . . 12

9 API Design **12**
- 9.1 REST Endpoints (FastAPI) . . . 12
- 9.2 Request/Response Schema . . . 12

10 Evaluation Strategy **13**
- 10.1 Evaluation Metrics . . . 13
- 10.2 Validation Approach . . . 13

11 Project Structure **14**

12 Risk Analysis **15**

13 Alignment with 2026 Governance Principles **15**

14 Conclusion **15**

15 References **16**

Executive Summary

ARIA (**A**daptive **R**eview **I**ntelligence **A**rchitecture) is an intelligent automatic code review system that accepts a GitHub repository URL as input and generates a comprehensive, structured code review report using multiple Large Language Models (LLMs).

The system addresses the critical 2026 industry challenge: the human capacity to review AI-generated and human-written code has not scaled with code generation tools, leading to "rubber-stamp" reviews, notification fatigue, and the accumulation of "AI Slop" — code that compiles but is fundamentally brittle and unmaintainable.

Key Innovation: The Model Debate Protocol

Unlike existing tools that use a single LLM or simply aggregate parallel outputs, ARIA implements a **Model Debate Protocol** where specialized agents from *different model families* independently review code, then *cross-verify each other's findings* through structured argumentation. Only findings that achieve multi-model consensus surface in the final report — eliminating false positives and hallucinations at their source.

Design Goals

G1. Multi-Model Diversity: Use genuinely different LLM families (Llama, Gemini, Mistral) to eliminate single-model bias.

G2. High Signal-to-Noise: Cross-verification ensures only confirmed, actionable findings reach the developer.

G3. Deep Context: Repository-level understanding via AST parsing and knowledge graphs, not just diff-level analysis.

G4. Structured Output: Severity-ranked, categorized findings with line-level annotations and fix suggestions.

G5. Zero Cost: Built entirely on free-tier APIs — no paid subscriptions required.

Problem Statement

The Industry Crisis

The software industry in 2026 faces a systemic quality bottleneck:

- **Code volume explosion**: AI copilots generate code at superhuman speeds, overwhelming human review capacity.
- **The "Savant Intern" paradox**: LLMs catch localized bugs but lack architectural understanding, generating 70–80% irrelevant comments.
- **AI Slop accumulation**: 38.1% of developers cite AI-generated technical debt as their primary concern.
- **Single-model limitations**: The c-CRAB benchmark shows even the best agents solve only ∼40% of complex review tasks.

- **No cross-verification**: Existing tools trust a single model's output without independent validation.

Problem Definition

> *Given a GitHub repository URL, design and implement a system that leverages multiple families of Large Language Models to perform automated, high-quality code review, producing a structured report with severity-ranked findings that have been cross-verified across models to maximize reliability and minimize false positives.*

Literature Review & Industry Analysis

Current Tool Landscape

Table 1: Leading Automated Code Review Platforms (2026)

Platform	Primary Innovation	Key Limitation
Claude Code Review	Dynamic multi-agent swarms scaling with PR size	Locked to Anthropic; $15–25/review
Qodo (CodiumAI)	Upstream policy enforcement via NVIDIA NIM + RAG	Enterprise-only; requires NIM infra
CodeRabbit	Multi-repo analysis; "Slow AI" methodology	Criticized for noise (Aiden Bai controversy)
BitsAI-CR (ByteDance)	Dual-stage RuleChecker + ReviewFilter	Proprietary; 219 hand-crafted rules
Reviewdog	Universal linter-to-PR bridge	No LLM intelligence; purely static
DeepAudit	Multi-agent vulnerability detection with PoC sandbox	Security-only; not general-purpose

Key Industry Trends

1. **LLM Pluralism**: Tools rapidly adding multi-model support — developers demand model portability and cost control.
2. **Repository-Level Context**: Moving from diff-only to full codebase understanding via knowledge graphs (e.g., GraphRAG achieving 6.8× token reduction).
3. **Agentic Review**: Evolution from "AI posts a comment" to "AI reviews, debates, and fixes."
4. **The "Slow AI" Movement**: Explicit rejection of pure velocity in favor of rigorous, deliberate review.

5. **Behavioral Metrics**: The "Outdated Rate" (ByteDance) and developer adoption rates replacing BLEU scores.

Gap Analysis

Table 2: Gap Analysis: Where ARIA Fits

Feature	CodeRabbit	Claude CR	BitsAI	ARIA
Multi-model families	×	×	×	
Cross-model debate	×	×	×	
Confidence scoring	Partial	Partial		
Full repo context (AST)	×	Partial	Partial	
AI-Slop detection	×	×	×	
Free / open-source	×	×	×	
Structured severity report				

System Architecture

High-Level Pipeline

The ARIA system operates as a five-stage pipeline, orchestrated by a LangGraph state machine:

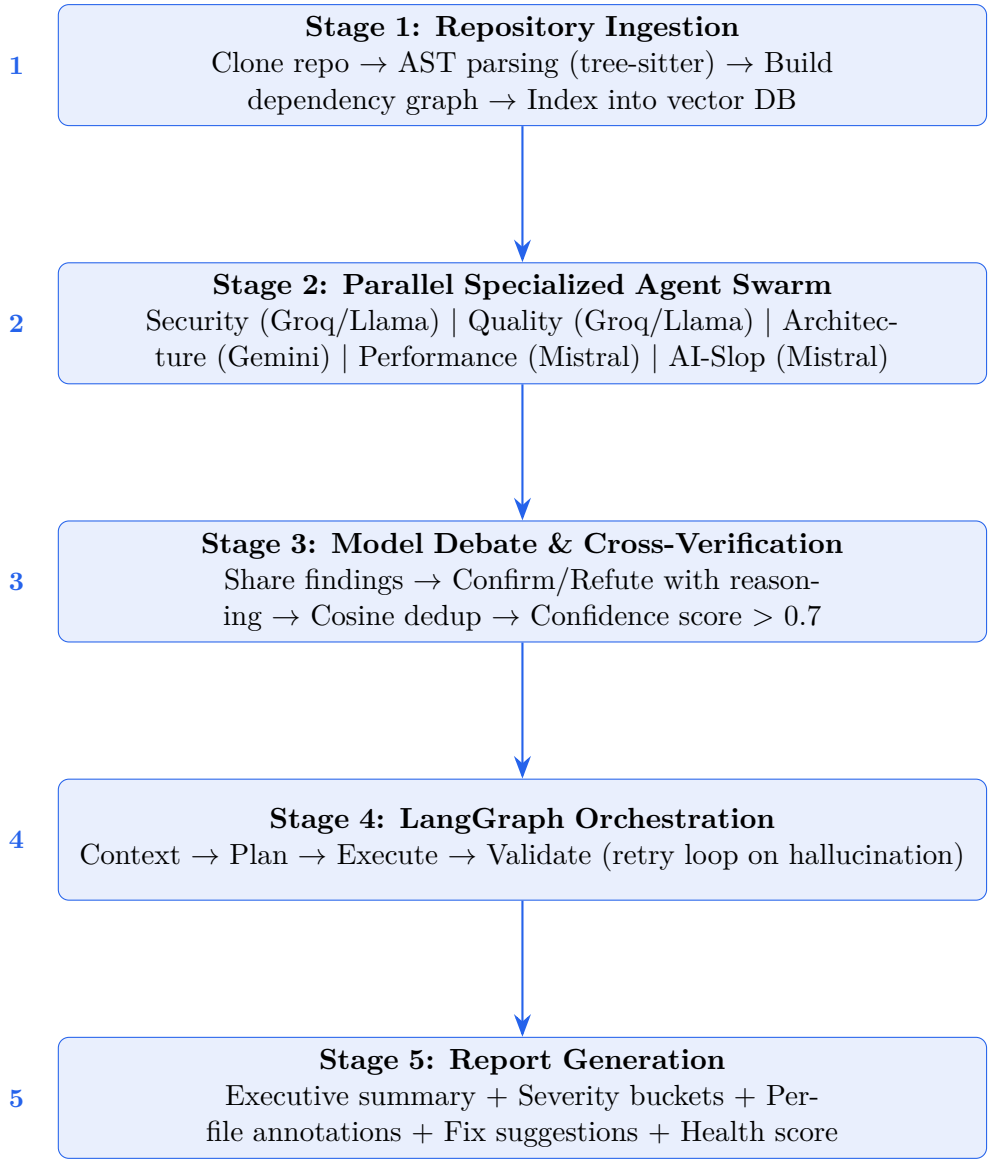

Figure 1: ARIA Five-Stage Pipeline Architecture

Detailed Component Architecture

Stage 1: Repository Ingestion Engine

- **Input**: GitHub repository URL
- **Clone**: Uses PyGitHub / `git clone` with depth limiting for large repos
- **AST Parsing**: `tree-sitter` extracts function signatures, class hierarchies, import graphs, and call relationships
- **Knowledge Graph**: `NetworkX` directed graph mapping function → function calls, class → method, file → imports
- **Vector Index**: `ChromaDB` stores code chunks with embeddings for RAG retrieval
- **Output**: Structured context package (file list, dependency graph, code chunks, metadata)

Stage 2: Specialized Agent Swarm

Five agents run in parallel, each with a different model family and specialized system prompt:

Table 3: Agent Specifications

Agent	Provider/Model	Focus Area	Review Dimensions
Security Auditor	Groq / Llama 3.3 70B	OWASP Top 10, CVEs, hard-coded secrets, injection attacks	Security
Code Quality	Groq / Llama 3.3 70B	Cyclomatic complexity, dead code, duplication, error handling	Maintainability
Architecture	Gemini 2.0 Flash	SOLID violations, coupling, design patterns, modularity	Structure
Performance	Mistral Small	$O(n^2)$ algorithms, N+1 queries, memory leaks, async issues	Performance
AI-Slop Detector	Mistral Small	Over-abstraction, unreadable patterns, brittle AI-generated code	Legibility

Stage 3: Model Debate Protocol

The core innovation. After parallel review:

1. Each agent's findings are serialized as structured JSON (file, line, issue, severity, reasoning).
2. Each finding is sent to **two other agents** from **different model families** for cross-verification.
3. Verifying agents respond with: `CONFIRM` (with additional evidence) or `REFUTE` (with counter-reasoning).
4. Findings are scored:
   - 3/3 confirm → confidence 1.0 (Critical/High)
   - 2/3 confirm → confidence 0.7 (Medium)

- 1/3 confirm → confidence 0.3 (filtered out)

5. Cosine similarity deduplication merges overlapping findings into a single representative issue.

The consensus formula:

$$C(f) = \frac{\sum_{i=1}^{n} v_i(f)}{n}, \quad v_i \in \{0, 1\}$$

Where $C(f)$ is the confidence score for finding f, v_i is the binary vote from agent i, and n is the total number of verifying agents. Only findings where $C(f) \geq 0.7$ appear in the final report.

Stage 4: LangGraph State Machine

The orchestration layer models the entire review as a directed graph with retry capabilities:

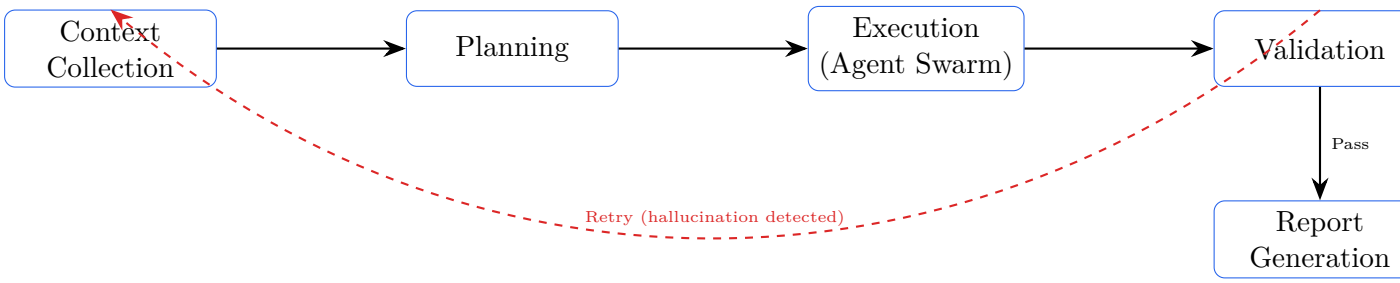

Figure 2: LangGraph State Machine with Retry Loop

Stage 5: Report Generator

The final stage produces a structured report with:

- **Health Score**: Composite 0–100 score based on weighted severity counts
- **AI-Slop Score**: Percentage of code flagged as potentially AI-generated and brittle
- **Severity Buckets**: Critical / High / Medium / Low with consensus scores
- **Per-File Annotations**: Line-level references with code snippets
- **Fix Suggestions**: Concrete code-level recommendations per finding
- **Output Formats**: HTML (via Jinja2), PDF, JSON

The health score is computed as:

$$H = 100 - (10 \cdot n_{\text{crit}} + 5 \cdot n_{\text{high}} + 2 \cdot n_{\text{med}} + 0.5 \cdot n_{\text{low}})$$

Clamped to $[0, 100]$.

Technology Stack

Table 4: Complete Technology Stack

Component	Technology	Purpose
Orchestration	LangGraph	State machine with retry loops
LLM: Security + Quality	Groq API (Llama 3.3 70B)	Fast, free inference
LLM: Architecture	Google Gemini 2.0 Flash	Free tier, strong reasoning
LLM: Performance + Slop	Mistral AI (Mistral Small)	Free tier, different family
AST Parsing	tree-sitter (Python bindings)	Language-agnostic AST extraction
Knowledge Graph	NetworkX	Dependency and call graph
Vector Store / RAG	ChromaDB	Code chunk embeddings for context retrieval
Deduplication	Sentence-Transformers	Cosine similarity for finding dedup
GitHub Integration	PyGitHub + subprocess (git)	Repository cloning and metadata
Report Rendering	Jinja2 + WeasyPrint	HTML and PDF report generation
Backend API	FastAPI	REST endpoints for the pipeline
Frontend UI	Streamlit	Web interface for input and report display
Language	Python 3.11+	Primary implementation language

Data Flow Diagram

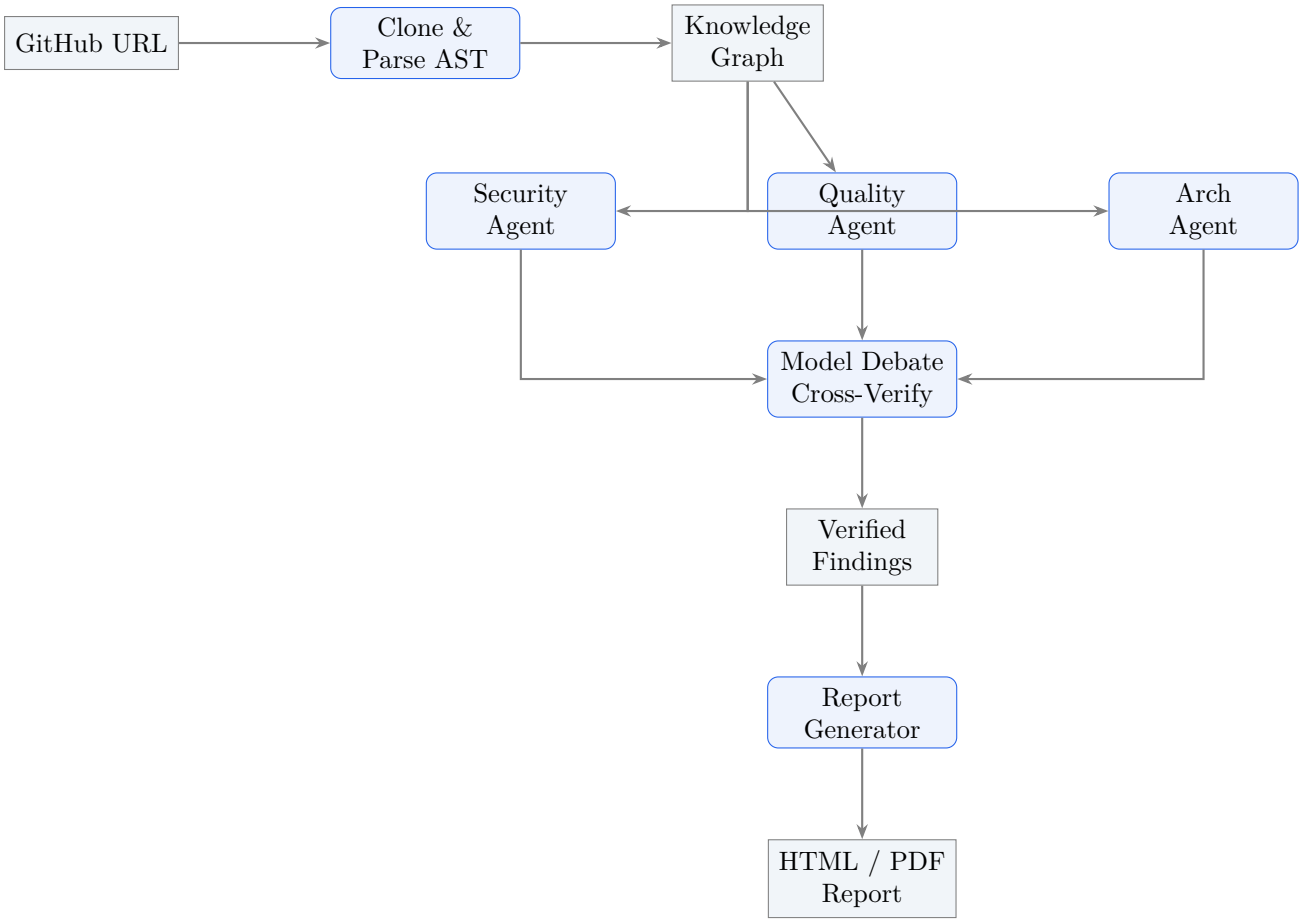

Figure 3: Data Flow through ARIA Pipeline

The Model Debate Protocol — Detailed Design

The Model Debate Protocol is ARIA's primary innovation. It addresses the fundamental limitations identified in the literature:

Motivation

- The c-CRAB benchmark (2026) shows individual agents solve only ∼40% of complex review tasks.
- AI agents and human reviewers highlight *fundamentally different* aspects of code.
- Different LLM families have different training data biases, failure modes, and reasoning patterns.
- Cross-verification is used in critical systems (e.g., dual-redundant avionics) for the same reason.

Protocol Steps

1. **Independent Review**: Each of the 5 specialized agents reviews the code independently, producing structured JSON findings.
2. **Finding Distribution**: Each finding is broadcast to 2 agents from *different model families* than the originator.
3. **Structured Argumentation**: Verifying agents must respond with:
   - `verdict`: CONFIRM or REFUTE
   - `reasoning`: Why they agree or disagree
   - `additional_evidence`: Any additional context they discovered
4. **Consensus Scoring**: Weighted voting produces a confidence score $\in [0, 1]$.
5. **Deduplication**: Sentence-Transformers cosine similarity groups semantically equivalent findings (threshold > 0.85).
6. **Filtering**: Only findings with confidence ≥ 0.7 pass to the report.

Why This Works

Table 5: Debate Protocol vs. Single-Model Review

Failure Mode	Single-Model	ARIA Debate Protocol
Hallucinated finding	Surfaces to developer	Refuted by 2 other models → filtered out
Missed vulnerability	Never caught	Another model family catches it independently
False positive	Wastes developer time	Cross-verification rejects unconfirmed findings
Model bias	Systematic errors persist	Different architectures have uncorrelated biases

AI-Slop Detection — Novel Contribution

ARIA introduces a dedicated **AI-Slop Detection Agent** — a novel contribution addressing the #1 developer concern in 2026 (38.1% of developers).

Detection Heuristics

The AI-Slop Detector evaluates code against these criteria:

1. **Over-abstraction**: Unnecessary wrapper classes, factory patterns for single-use objects
2. **Cargo-cult patterns**: Design patterns applied without justification (e.g., Strategy pattern for one implementation)

3. **Unreadable composition**: Deeply nested lambda chains, point-free style obscuring intent
4. **Suspicious uniformity**: Identically structured code across unrelated modules (copy-paste from AI generation)
5. **Missing error paths**: Clean happy-path code with no edge case handling
6. **Comment-code mismatch**: Docstrings that describe generic behavior rather than actual implementation

AI-Slop Score

$$S_{\text{slop}} = \frac{\text{Files flagged with AI-Slop indicators}}{\text{Total files analyzed}} \times 100$$

- $S < 15\%$: **Low risk** — codebase appears human-authored and intentional
- $15\% \leq S < 35\%$: **Moderate risk** — some patterns warrant review
- $S \geq 35\%$: **High risk** — significant AI-generated technical debt detected

API Design

REST Endpoints (FastAPI)

Table 6: ARIA API Endpoints

Method	Endpoint	Description
POST	`/api/review`	Submit a GitHub URL for review; returns a job ID
GET	`/api/review/{job_id}`	Get the status and results of a review job
GET	`/api/review/{job_id}/report`	Download the final report (HTML/PDF/JSON)
GET	`/api/health`	System health check and API connectivity status

Request/Response Schema

Listing 1: Review Request

```
{
    "repo_url": "https://github.com/user/repo",
    "branch": "main",  // optional
    "review_depth": "standard" | "deep",
    "output_format": "html" | "pdf" | "json"
}
```

Listing 2: Review Response

```
{
    "job_id": "aria-20260401-abc123",
    "status": "completed",
    "health_score": 67,
    "ai_slop_score": 23.5,
    "findings": {
        "critical": [...],
        "high": [...],
        "medium": [...],
        "low": [...]
    },
    "report_url": "/api/review/aria-20260401-abc123/report"
}
```

Evaluation Strategy

ARIA adopts a multi-layered evaluation approach informed by the 2026 consensus that lexical similarity metrics (BLEU, chrF) are obsolete for code review assessment.

Evaluation Metrics

Table 7: ARIA Evaluation Framework

Metric	Methodology	Target
Precision	% of findings confirmed as true issues by manual audit	> 70%
Consensus Quality	% of findings that survive cross-verification	> 60%
False Positive Rate	% of findings flagged as incorrect by human review	< 15%
Coverage	% of known issues (from seeded bugs) detected	> 50%
Debate Effectiveness	Reduction in false positives: debate vs. no-debate	> 40% reduction

Validation Approach

1. **Known-Bug Repositories**: Test against open-source repos with documented CVEs and known bugs.
2. **A/B Testing**: Compare single-model review vs. debate-based review on the same repos.
3. **Cross-Family Analysis**: Measure which model families catch which categories of issues.

Project Structure

Listing 3: Project Directory Layout

```
Automatic-code-reviewer/
|-- .env                          # API keys (gitignored)
|-- requirements.txt              # Python dependencies
|-- README.md                     # Project documentation
|-- docs/
|   |-- design_document.tex       # This document
|   |-- design_document.pdf       # Compiled PDF
|-- src/
|   |-- main.py                   # FastAPI application entry point
|   |-- config.py                 # Configuration and environment
    loading
|   |-- ingestion/
|   |   |-- github_client.py      # GitHub cloning and metadata
|   |   |-- ast_parser.py         # tree-sitter AST extraction
|   |   |-- knowledge_graph.py    # NetworkX dependency graph
|   |   |-- vector_store.py       # ChromaDB indexing
|   |-- agents/
|   |   |-- base_agent.py         # Abstract base agent class
|   |   |-- security_agent.py     # OWASP/CVE focused agent
|   |   |-- quality_agent.py      # Code quality agent
|   |   |-- architecture_agent.py # Architecture review agent
|   |   |-- performance_agent.py  # Performance analysis agent
|   |   |-- slop_detector.py      # AI-Slop detection agent
|   |-- debate/
|   |   |-- debate_manager.py     # Cross-verification orchestrator
|   |   |-- consensus.py          # Confidence scoring and dedup
|   |-- orchestrator/
|   |   |-- graph.py              # LangGraph state machine
|   |   |-- states.py             # State definitions
|   |-- report/
|   |   |-- generator.py          # Report compilation
|   |   |-- templates/            # Jinja2 HTML templates
|   |   |-- scoring.py            # Health score computation
|-- app/
|   |-- streamlit_app.py          # Streamlit web UI
|-- tests/
|   |-- test_agents.py            # Agent unit tests
|   |-- test_debate.py            # Debate protocol tests
|   |-- test_pipeline.py          # Integration tests
```

Risk Analysis

Table 8: Risk Matrix

Risk	Likelihood	Impact	Mitigation
API rate limiting	High	Medium	Implement exponential backoff; cache responses; rotate between providers
Gemini quota exhaustion	Medium	Low	Fall back to Groq with different system prompt
Large repo token overflow	Medium	High	tree-sitter context windowing; chunk files; prioritize changed files
Hallucinated findings	Medium	High	Model Debate Protocol eliminates via cross-verification
Slow network / timeouts	Medium	Medium	Async execution with configurable timeouts; partial report generation

Alignment with 2026 Governance Principles

ARIA explicitly implements the four principles of AI Code Governance:

1. **Comprehension as Requirement**: The AI-Slop Detector enforces legibility standards — code that cannot be understood by humans is flagged, regardless of functionality.

2. **Responsibility Boundary**: The models that generate findings are never the same models that verify them — strict separation via the Debate Protocol.

3. **Early Visibility of Risk**: The Knowledge Graph and full-repo context surface architectural risks before they become production incidents.

4. **Preservation of Human Discernment**: ARIA is strictly advisory — it generates reports, not approvals. The human developer retains full authority.

Conclusion

ARIA represents a novel approach to automated code review that addresses the critical industry challenges of 2026. By combining multi-LLM parallel analysis with a structured cross-verification debate protocol, the system produces higher-confidence, lower-noise review reports than any single-model approach. The inclusion of a dedicated AI-Slop detector directly addresses the #1 developer concern, while the full-repository knowledge graph ensures reviews are contextually grounded.

The system is fully feasible using free-tier APIs (Groq, Gemini, Mistral), making it accessible for academic research and open-source deployment without requiring enterprise subscriptions.

References

[1] ByteDance, "BitsAI-CR: Industrial-Scale Code Review," *arXiv preprint*, 2025.

[2] Anthropic, "Claude Code Review — Dynamic Multi-Agent Architecture," 2026.

[3] Qodo, "The Four Principles of AI Code Governance," *Qodo Blog*, 2026.

[4] Code Review Agent Benchmark (c-CRAB), "Executable Evaluation for Review Agents," 2026.

[5] Alibaba, "AACR-Bench: Repository-Level Code Review Benchmark," *GitHub*, 2026.

[6] tirth8205, "code-review-graph: Knowledge Graph for Code Review," *GitHub*, 2026.

[7] IEEE Study, "Impact of AI Code Review on Developer Behavior," *IEEE Software*, 2026.

[8] LangGraph Documentation, "State Machine Orchestration for LLM Agents," 2026.

[9] OWASP, "Top 10 Web Application Security Risks," 2025 Edition.

[10] Harrison Chase, "Context Engineering for AI Agents," *LangChain Blog*, 2026.